Where Does the Cost of Verifying AI-Generated Formal Artifacts Come From?

Anonymous research draft

Abstract

AI systems produce mathematical libraries, verified programs and streams of proof candidates. Their verification costs span library loading, elaboration, proof construction, kernel reduction and operating-system work. We characterize these costs, using the research-level `openai/math` library as the primary workload and retained proof/program candidate streams as contrasts. A complete source census covers 163,097 Lean modules; bounded native measurements retain successful, failed and dependency-blocked work. In 26 randomly selected modules compiled with standard compiler artifacts, system CPU accounts for 28.26% of measured compiler CPU. A large Grid module records 45.38 million voluntary context switches and averages 1.33 CPU cores despite eight requested workers. The default root omits 33.30% of the release's lexical `decide +kernel` occurrences. Budget-failure exposure varies across candidate streams, and profiling can change resource-bounded outcomes: seven of 200 diagnostic source pairs change error flags. We distinguish these observations from semantic falsity, whole-build costs and accelerator opportunity. [**RESULT: Public generated-software output characterization, complete cost-estimation coverage, calibrated import attribution and system-CPU causes.**] Certificate-repair experiments illustrate when changing proof representation can help pathological reductions and when it adds cost. The contribution is a characterization and its implications for verifier design, rather than a new branching service or certificate algorithm.

1 Introduction

AI-generated formal artifacts now range from individual proof attempts to research mathematical libraries and multi-module verified programs. Verification supplies the acceptance signal for these artifacts, but the cost of obtaining that signal is not a single solver call. Lean must prepare imported declarations, elaborate source, execute tactics, construct proof terms and check them in the kernel [4]. SMT-backed program verification adds source translation and solver interaction [8]. The cheapest stage to optimize may account for little of the complete cost.

This paper asks where that cost comes from. Our primary workload is the released `openai/math` research library [24], rather than treating short contest proofs as representative of all generated formal artifacts. We separately examine the availability of generated software repositories and use miniF2F, ProofNet and AutoVerus candidate streams as contrasts. We use existing released outputs; we do not generate a new evaluation corpus. Repository scaffolds, reference proofs and reported scores alone do not establish that evaluated model outputs are available.

Our central observation is that verification exposes different bottlenecks at different boundaries. Source-library compilation has substantial system CPU and repeated process-level preparation. Some candidate streams spend most of their measured work on requests that end at resource limits; others have a much smaller slow tail. Profiling can itself alter those limits. Even the selected build domain changes the apparent amount of computational proof work. Consequently, a lexical tactic count, a successful compiler subset or an operation-only speedup cannot stand in for verification cost.

We organize the study around four questions:

RQ1 How do artifact granularity, released-source coverage and build scope change the verification workload being measured?

RQ2 How do imports, source processing, proof construction, kernel checking and operating-system work contribute to CPU, latency and memory?

RQ3 How do failures, resource limits and observation methods change cost distributions and the reproducibility of verifier outcomes?

RQ4 Which system-design implications survive complete verification, and where do targeted proof-representation changes fail to generalize?

Each finding section presents its evidence, measurement boundary and implication. We do not assume that all verification workloads need the same cache, scheduler, proof representation or accelerator. Our contributions are a source/build-domain characterization, a boundary-aware account of measured native costs and outcomes, and case studies testing the resulting design implications. Missing results remain explicit; the current draft does not claim a complete full-library build or completed generated-software evaluation.

2 Methodology

2.1 Workload selection and artifact provenance

Our primary population is the complete `openai/math` release at `4eaee87548250b2682affda7d05d947c05554bae` [24]. It contains 163,097 Lean files and 1,785,499,363 source bytes. We pin Lean 4.34.1, Mathlib `d13f23b7` and the manifest's external dependencies. Version-matched prebuilt dependency caches prepare the context; their acquisition is not silently included in target compiler CPU. Required dependency compatibility patches and retained objects are recorded as part of that context, rather than treated as candidate-source changes. A source module, its prerequisite closure and the entire library are different units. Source-directory overrides are resolved before computing internal import reachability.

We contrast research mathematics with public generated-software artifacts and retained candidate streams. Availability is checked at an immutable repository revision: a benchmark task, score or reference implementation does not substitute for the model's submitted source. We preserve declared specification axioms, admissions and incomplete obligations, and never generate replacements for unavailable outputs. The software admission audit is detailed below.

The candidate contrasts retain the original 7,808 miniF2F sources, 5,952 ProofNet sources per recorded pass, and 3,566 AutoVerus outputs, including unsuccessful and intermediate attempts [23]. EPFL and APRIL provide separately labelled feedback/mutation/reference subsets [5, 34]. Source-sorted replay is not a historical arrival trace. Selection, toolchain, context, native libraries and limits accompany each point; different streams are not pooled into one distribution. Generation costs are excluded except where explicitly reported as publisher log accounting. All-candidate verification and first-success consumption have different stopping rules.

2.2 Stratified sampling and nonresponse

The probability frame is all 163,097 released modules, including those outside the default `OAI` root. We cross four source-size percentile bins (0–50, 50–90, 90–99, 99–100), three internal import-depth bins (1–10, 11–100, 101+), and presence of comment/string-filtered `decide +kernel`. There are 21 nonempty cells. With seed 20261030, we draw two modules uniformly without replacement in each cell, giving 42 original targets. A module in stratum h of size N_h has inclusion probability $2/N_h$. A two-module stratum is a census. Purposeful large/deep/computation stresses and prerequisites get no additional population weight.

Targets are compiled after their prerequisite objects exist. Dependencies, cache preparation and target compilation retain separate records rather than pretending that a target's local CPU is its complete cold-build cost. Expensive or blocked draws stay in the original frame; we do not replace them with easier modules. At the legacy standard-artifact point, 26 targets have complete costs. The first locked continuation, five unprofiled panel baselines and the collected cohort C below increase the accumulated matching-native-setting inventory to 29/42 target costs and 12 complete strata. Its strict shared-lock subset has eight costs and three complete strata. Legacy and locked timing policies stay separate; neither is a completed same-protocol full-population sample. Dependency compiles and repeated profiles do not become additional probability draws.

For a fully observed stratum of size N_h with $n_h = 2$ planned draws, we use the expansion estimate and finite-population variance

$$\widehat{T}_h = N_h \bar{y}_h, \qquad \widehat{V}_h = N_h^2 (1 - n_h/N_h) s_h^2 / n_h,$$

where y is measured user plus system CPU and s_h^2 is sample variance. These are compiler-CPU totals conditional on already-built imports and the stated output facets. We do not replace n_h by a completed-response count in an incomplete cell. Nominal normal-cost t intervals from two observations are fragile under heavy tails and exclude runtime/systematic variation. Whole-library totals and intervals remain unavailable until the required draws have valid matching costs. A default-root domain total uses the original inclusion probabilities and response $y_i I(i \text{ in root closure})$; known domain exclusion is not a measured zero-cost compilation.

2.3 Platform and serialized native measurements

The current node is phobos-vt, an Intel Core i9-14900K machine. The inspected Linux PMU CPU sets identify P-core CPU IDs 0–7 and E-core IDs 8–23; observed physical RAM is 64,205,640 KiB. New timed native compilation/profile captures take one shared exclusive P-core lock and use affinity 0–7. Untimed preparation and root analysis use E cores. This serializes measurement captures across the participating lanes; it does not mean Lean is single-threaded. The library compiler requests eight workers per native process. We report actual CPU/wall ratios rather than equating eight workers with eight utilized cores.

The current source-cost point emits `.olean`, `.ilean` and `.c`, preserves authored heartbeat/recursion options and uses `autoImplicit=false` and the 9,216 MiB Lean memory option. Earlier olean-only, different-budget and pre-lock points remain separately labelled. Pre-lock observations are not relabelled as serialized controls after the privileged restart. Timed captures retain their own finite group/per-file allowances, memory records, source identity, actual arguments and PID/birth/terminal evidence. Lean memory checks and one-second RSS re-

sponse policies are not kernel hard quotas. A later deployment change does not rewrite an earlier measurement's conditions.

Library objects and raw captures reside on the existing PVC. No cache drop is performed. The import matrix is sequential with warm OS caches and fixed build-then-import ordering; it is not a randomized cold-storage experiment. A native compile is counted as such only after exit zero without a sorry warning. That criterion is not a whole-repository axiom-closure audit. Software corpora retain their own SDKs and imports: the core/Std-only LKV boundary is not forced through a Mathlib-loaded service.

2.4 Timing, profiling and outcome comparison

Native elapsed time runs from process launch to observed exit/collection; user/system CPU and leader high-water RSS come from native process accounting. Group elapsed time additionally charges between-job handling and retirement; external capture time includes the controlling lifecycle. Queue time before acquiring the shared timing lock is separate. A concurrent service makespan, summed physical attempts, a frontend-body timer and an extra warm declaration check keep their own denominators.

The import matrix has 13 modules selected by historical build time below five seconds within the 26 completed targets. Each receives three adjacent build/import-only pairs, with all compiler output facets retained; three Mathlib-only controls are additional. Its 100 ms memory observer has material CPU cost, recorded separately. Sampled memory components are taken from the same per-run RSS-peak sample; medians across runs are not combined into one fictitious process. Import-only controls remove declarations, not proof obligations from the reported original builds. Differences are descriptive preparation exposure, not a proved removable fraction.

Unprofiled timing and perf-record executions are separate conditions. Source, imports, toolchain, budgets, worker settings and output facets are matched before comparing them. Hardware-event samples are interpreted within their event and privilege scope: user-only cycles do not attribute kernel CPU, and overlapping cumulative phase timers do not form a wall-time partition. Stacks, missing/unresolved samples and observer CPU remain visible. We check diagnostics and resource outcomes as well as successful completion; a shorter profile ending at an earlier budget error is not a speedup. **[RESULT: Completed native system-CPU attribution, matched profile perturbation controls and calibrated low-overhead import results.]**

Rechecking compiled declarations is a distinct experiment. Import-relative `leanchecker` trusts imported declarations and replays the target; `--fresh` includes imported constants. The former is not a whole-closure independent implementation. An extra warm check also excludes original proof construction and first checking. Compiler, replay CLI and kernel-only subclocks are never used interchangeably [17, 3].

2.5 Censoring and explicitly conditional projections

A native timeout, watchdog interruption, allocator diagnostic, ordinary error, platform interruption and never-invoked dependency are distinct. A group deadline can give the last job less time than its nominal per-file allowance; its actual allowance is retained. Platform-interrupted work with unknown cost is not fabricated as a native timeout. Neither budget exhaustion nor an ordinary failed tactic establishes that its mathematical goal is false.

Static source/token counts and graph depths are exact at their lexical boundary, not runtime calls or timed spans. Earlier per-file/byte-rate cost models and 8/24-core schedules are hypothetical. Scaling selected import medians by the library's module count is not a full-build estimate. Observed successful artifact/source byte ratios are not population-weighted storage forecasts or bounds on live RAM. We retain failed and uncompleted samples rather than reporting a successful-subset global estimate. **[RESULT: Complete all-source/default-root native totals and uncertainty, cold preparation/linking costs and generated-software measurements.]**

2.6 Generated software requires released outputs

Table 1 separates publicly described tasks from usable generated artifacts. Vero describes 43 multi-module Lean 4 instances and joint code-and-proof and proof-only modes [37]. Its 43 active instances contain 640 Lean scaffold files; their within-project import depth reaches 35, excluding external dependencies. The pinned public tree and inspected release assets do not establish release of the evaluated agent-run outputs. A historical 13-file BankLedger solution has unestablished model provenance. The live project site returns HTTP 403 from the measurement Workspace, so this is a scoped repository audit, not a claim that outputs are absent everywhere. We do not replace model outputs with references or run new generation.

Vericoding supplies an actual public model-output corpus, beyond task specifications [2, 35]. The data repository's 626 Lean programs all occur in the code repository's 1,503 model-tagged Lean snapshots (1,490 distinct source byte strings). We do not add those overlapping collections. The same code release exposes 45 detailed Verus/Dafny final-result tables: 16,025 outcome rows contain 14,109 nonempty final-program fields.

Software source	Public-artifact evidence	Admission and current measurement scope
Vero	43 multi-module instances described; inspected repository `0a7325df` contains scaffolds and reference material	Evaluated agent outputs not established by this audit; native generated-output costs [**RESULT: pending output provenance**].
Vericoding	Data `387cd699` and code `b265a3b2` release 1,503 model-tagged Lean snapshots; the original 626 are a subset. There are also 45 Verus/Dafny final-result tables	Released outputs are available, with partial publisher joins; complete candidate trajectories, native acceptance and per-stage costs [**RESULT: pending**].
Lean Kernel Verify	Pinned `129e65bf` publishes Lean checking code and KIR fixtures, using `native_decide`	A released checker/fixture workload; AI-authorship/output provenance and native repository costs [**RESULT: pending**].

Table 1: Software-workload availability audit, not completed verification. No candidate is generated to fill a missing release. Public files and publisher outcomes do not establish accepted, axiom-clean proofs.

These are output occurrences, not 14,109 distinct programs or all intermediate repair attempts. All explicit `error_message` fields in these 45 tables are empty; schema fields and prompt/debug references do not provide a complete native diagnostic history. Separate outcome-only tables have 55,397 final records but no program bodies or per-request durations; they cannot supply a verification-time denominator.

For the Lean snapshots, task-ID/model joins associate 1,354 files with a publisher success and leave 149 without a matching row. Such associations are neither our native verdicts nor a soundness audit. The masked-source scanner marks `axiom` in 35 files and `sorry` in 12; these lexical flags require a native declaration/axiom audit before classifying accepted artifacts. All admissions and specifications must remain part of the replay context. The code repository declares Lean 4.23.0-rc2 and Mathlib `90c0e186`, but these do not establish each historical run's exact environment. Original Verus compiler builds remain unknown.

The published lexical scan finds `simp` in 1,220 of the 1,503 Lean snapshots and `rfl` in 693, with no `bv_decide` or `decide +kernel` identifiers. These are comment/string-masked source occurrences, not executed tactic counts or CPU fractions. Each standalone snapshot has within-input import depth one; its external Mathlib/Std dependency closure is excluded. Thus the available software candidates complement the deep computational mathematics library, but do not yet show which verifier stage dominates. Native measurement must preserve per-language versions, output provenance and repository-level outcome boundaries. [**RESULT: Toolchain-qualified software builds, axiom/admission audit, per-stage native costs and complete candidate-stream coverage.**]

Lean Kernel Verify is a distinct software verification boundary: a frontend lowers compiler IR to KIR and its proof-generation template uses `native_decide` for concrete-case properties [22]. The public input set has 90 KIR fixtures and 12 Lean checker files; it is not an identified LLM candidate stream. The identifier occurs in a template string rather than masked executable Lean source, so a raw text hit cannot be reported as a measured tactic invocation. Its published checker can prove a safety violation; such a result is not a Lean proof failure. The public README states that some full frontend paths still require its originating monorepo. Released fixtures are useful characterization inputs, but do not by themselves establish availability of the generated software histories sought here. We keep these provenance and trust distinctions explicit.

Domain	Modules	Kernel-decision tokens
All released source	163,097	1,354,379
Default-root closure	130,834	903,371
Outside default root	32,263	451,008

Table 2: Comment/string-filtered `decide +kernel` occurrences. The 33.30% omitted share is lexical, not a runtime or native-check share.

3 The default build omits a third of computational proof sites

The released library and its default build root are different workload domains. Resolving the publisher's source-directory overrides and imports, the default `OAI` root reaches 130,834 of 163,097 modules. The remaining 32,263 modules contain 387,698,422 source bytes.

All 562 PlanarPacking and 466 Transonic modules lie outside that default root at the pin. They contain 255,199 and 193,076 such occurrences. Thus a default-root build can omit precisely the computational proof families that motivate kernel-optimization studies. Conversely, explicitly building those sources is a workload extension, not evidence about the default command's cost.

Lexical density also overstates what is known about repeated kernel work. In 363 retained compiled modules, 45,661 auxiliary declarations match the decision-helper shape, with 42,610 globally distinct expression-type keys. The corresponding source-site count is 42,886: one source site can expand to several helpers, and im-

Completed cohort	Modules	Aggregate ratio	Maximum
A: original targets	26	54.697	230.725
B: prerequisites	61	42.959	314.668

Table 3: Observed compiler-output/source byte ratios, not CPU cost or whole-library storage predictions. Failed/interrupted outputs are excluded from these success-only ratios and retained separately.

ported/cached objects use another boundary. Lean's auxiliary-lemma cache is branch-local [15, 14]. Neither global source-text equality nor equal helper types establishes cache misses, full dependency equality or seconds saved.

Compiler outputs are a separate size boundary. Completed native records show that source bytes can badly understate the artifact footprint (Table 3). The 26 original probability targets produce 77,581,124 output bytes from 1,418,376 source bytes. A separate 61-module completed prerequisite cohort produces 420,157,676 bytes from 9,780,494 source bytes. Each total counts exactly one `.olean`/`.ilean`/`.c` set per completed module, not toolchains, external caches or duplicated capture copies.

For example, a 9,339-byte `Shortening` module yields 2,938,688 artifact bytes, of which 2,893,832 are olean bytes. This identifies serialized output to inspect; it does not establish proof-term duplication or the CPU fraction spent exporting it. Both aggregate ratios exceed the earlier planning assumption of 40 times, so that assumption is not a safe storage upper bound. Neither successful cohort justifies multiplying its ratio by all released source bytes or estimating peak memory from source size.

System-design implication. A representative verification benchmark must specify both the released-source domain and the exact build target/artifact set. A compiler or cache evaluated only on the default root cannot claim coverage of omitted computational families. Population estimates must retain the original sampling probabilities when reporting a default-root subdomain; known out-of-domain membership is not a measured zero-cost compilation. Build schedulers and artifact caches should therefore expose the dependency closure and emitted outputs as part of the workload contract, so that a faster default target is not mistaken for faster verification of all released artifacts.

4 System CPU can dominate without using the requested cores

Across the 26 completed standard-artifact probability targets, measured user CPU is 510.823 seconds and system CPU is 201.187 seconds. System work is therefore 28.26% of the 712.010-second CPU sum. This is the aggregate of completed random draws, not a population-weighted full-library fraction: 16 selected draws remain incomplete at this point. Within those 26 observations, two modules, `UMoments9` and `BarrierData41`, account for 84.11% of the measured system CPU; adding `Barrier50` raises the share to 89.39%. Their respective system fractions are 44.33%, 38.91% and 41.56%. In contrast, `EnvelopePsiB` peaks at 8.10 GiB leader RSS with a 3.82% system fraction. High resident memory alone does not explain this variation. This is a descriptive comparison at the old point, not a new perf attribution or a population-weighted ranking.

A later strictly locked continuation returns 14 native invocations: 13 compile and the last is interrupted by the remaining group budget. Their summed CPU and native wall times are 3,304.393 and 1,700.018 seconds, or 1.944 average native CPU cores; group and external capture clocks are 1,700.148 and 1,701.050 seconds. This is a bounded prefix of 659 planned jobs for six endpoints, not their complete build. Only `CoverageK14` reaches an original probability endpoint, taking 11.016 seconds locally after prerequisites. At that boundary, the combined inventory reaches 27/42 target costs and strict locked coverage is one; the separate five-case panel below later raises strict coverage to six; cohort C below raises it to eight. Full total CPU and a global interval remain [**RESULT: unavailable from the incomplete matching sample**].

The freshly re-executed `CertificatesK14Part003` compiles in 258.741 seconds, but its earlier platform-interrupted cost is unknown; no before/after speedup follows. The final `KernelValues20` receives a 92.930-second remainder and returns after 93.078 seconds, rather than using a complete 600-second opportunity. Fourteen source/PID/birth records are checked and retired. These costs retain the deadline and import-cache boundary; they do not partition system CPU into causes.

A second completed bounded capture, cohort C, returns 544 native calls: 543 compile with the three standard outputs and one reaches the group deadline; 73 of its 617 planned jobs never start. Only two original probability endpoints finish, taking 3.946 and 1.644 seconds locally. The remaining completed calls are dependency work, not 541 additional costed draws. Its summed CPU and native wall clocks are 2,077.369 and 1,699.465 seconds, or 1.222 average native CPU cores; group elapsed time is 1,700.033 seconds. The final dependency runs for 0.335 seconds with only 0.303 seconds of group allowance.

It is not a failed 600-second opportunity. Source bytes, successful artifact sizes and all 544 native identities are checked. Current coverage is 29/42 under matching native compiler settings, or 8/42 with the strict shared lock; neither completes the original sample.

The purposeful Grid module exposes a more extreme operating point. It compiles in 634.280 elapsed seconds with 399.934 user and 440.563 system CPU seconds. Its 52.42% system share accompanies 45,375,721 voluntary and 529,643 involuntary context switches. Total CPU divided by wall time is 1.325 cores despite eight requested Lean workers. There are no observed major page faults. These counters establish low realized CPU utilization and substantial system work; they do not identify a lock, allocator, futex or scheduler as the cause.

Another large certificate module, Amplitudes, uses 228.898 user and 95.954 system CPU seconds over 180.276 elapsed seconds: 29.54% system CPU and 1.802 average cores at its separate point. Thus the inspected large modules do not all share a 39–52% range. [**RESULT: Matched certificate-module cohort and its system-share distribution, perf attribution to synchronization/allocation/I/O, and causal controls.**] The earlier one-worker profiled Grid run stops at its own 300-second bound; it cannot supply an isolated eight-versus-one-worker speedup.

Native profiles expose different work inside the same compiler. A separate five-source panel brackets perf-stat and perf-record runs with unprofiled before/after baselines: 20 native invocations in all. The two mathematical comparators are the original random pilots in one high-variance stratum; three heavy original targets are selected for mechanism inspection. The panel has no population phase weights. Every source compiles in all four modes with exact source bytes and identical native diagnostics, and every recorded native process retires.

Figure 1 (right) attributes native-PID user-cycle sample periods to the nearest recognized caller family. Every hotspot percentage below is a sampling distribution within that event and PID scope, not a percentage of complete-process elapsed time or total user-plus-system CPU. The three computational cases have 74.93–94.01% in the C++ checker family; the two mathematical comparators have only 1.92–2.06%. The C++ family includes inference, equality, reduction and checker-state cleanup, rather than solely final theorem validation. The contrast supports measuring mathematical and computational proof families separately; it is not an all-library kernel fraction. Within the same sampled user-cycle denominator, normalization anchors account for 71.43%, 65.71% and 80.61% in the three computational cases; checker-state cleanup accounts for another 9.30%, 8.87% and 13.34%. These symbol subfamilies explain the breadth of the checker label without assigning a source-level tactic from a C++ frame. Private-name recognition refines the initial Meta/import attribution; it changes classification, not the original native recordings.

The five recordings contain 46,882 native samples and zero reported lost samples. Their decoded period totals plus excluded launcher periods match stock perf-report totals. Zero lost samples does not guarantee complete stacks: full root markers are rarely reached, and the shortest recording has only 215 native samples. Stat IPC spans 1.450–2.189 and cache misses per thousand instructions 1.379–6.101, but those counters include the launcher and native descendants; the phase table separately filters to the native PID. Per-case baseline-after CPU differs from baseline-before by -9.91% to +6.38%. One bracket per source cannot establish a general overhead correction or interval, even with equal observed diagnostics. System CPU remains outside the user-cycle phase denominator. This exclusion matters even among the three computational cases. In their unprofiled before baselines, UMoments9 and BarrierData41 spend 45.33% and 39.73% of native CPU in system mode, with approximately 115,680 and 131,232 voluntary context switches per wall second. EnvelopePsiB instead spends 4.12% in system mode and records 572 such switches per second. The two mathematical comparators have system shares 10.73% and 19.54%. Each fraction uses that invocation's own CPU denominator; switch counts alone do not attribute the work to a syscall. UMoments9 and BarrierData41 are outside the publisher's default import closure, whereas the other three panel sources are inside. Consequently, a default-root-only profile would omit two of these expensive computational cases, even before weighting or extrapolation.

[**RESULT: Causal attribution of the complete modules' system CPU and repeat-based profile-overhead uncertainty.**]

System-design implication. Worker count is not available parallelism. Increasing executor threads or offloading arithmetic can leave synchronization, allocation and kernel entry costs unchanged. The large system shares make allocator behavior, runtime memory accounting and synchronization concrete optimization candidates. Preserve proof checks while testing those runtime changes. System time and context switches alone do not identify which path causes the cost; the low-system EnvelopePsiB case also rules out treating every large computational module alike. Measure user/system CPU, elapsed time and actual runnable work together, then use matched causal controls before claiming a gain. User-cycle sampling can prioritize normalization and cleanup experiments, but cannot explain the excluded system work or establish a whole-flow accelerator ceiling. No runtime bottleneck is yet claimed fixed.

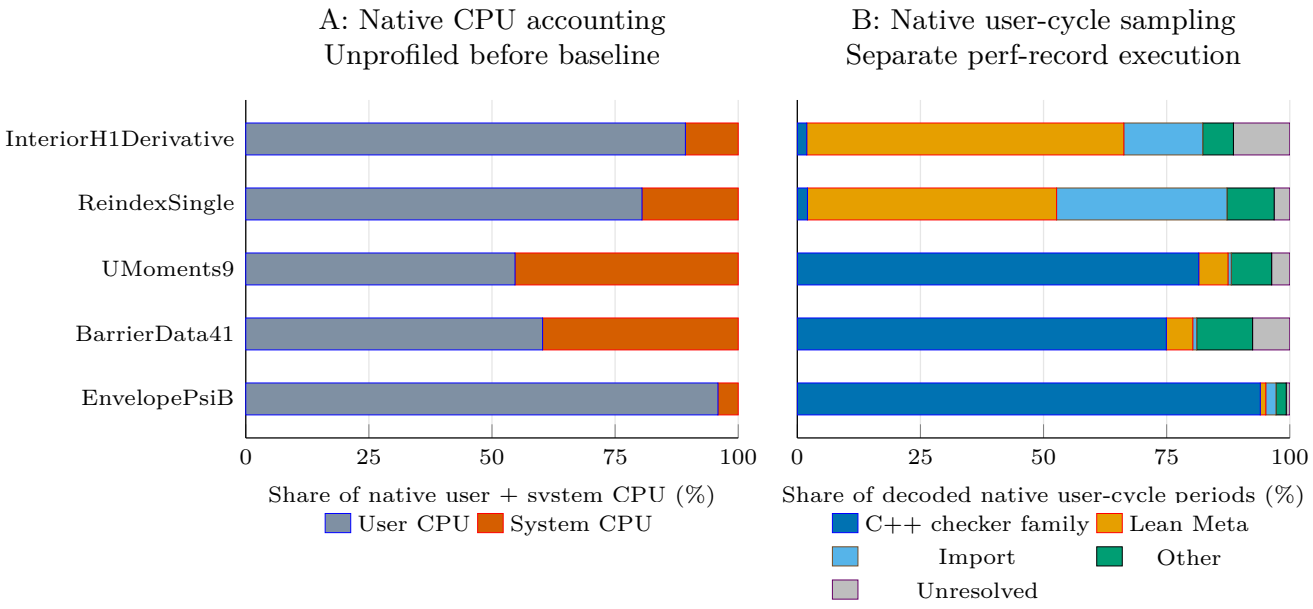


Figure 1: The same five purposeful sources have different cost mixtures. Left: native CPU accounting in each unprofiled before baseline. Right: decoded native-PID user-cycle sampling in a separate perf-record execution, using published revision-two caller-family classification. These are separate denominators and runs; the panels are not multiplied into a full-process phase estimate. The checker family includes normalization/inference/equality and cleanup, not only final validation. Other includes recognized elaborator/compiler/serialization families; unresolved includes runtime leaves without a recognized phase anchor.

5 Fresh module verification has a preparation floor

Each separately launched module compiler reconstructs its imported environment, even when dependency objects already exist. In the 26-module standard-artifact sample, the fastest complete compiler call takes 1.116 seconds and peaks at 2,598,928 KiB (2.48 GiB) leader RSS, for a 2,072-byte source. This is a measured whole-process floor in that sample, not an isolated Mathlib import timer or a universal minimum. The clock includes source processing and artifact emission.

A completed import-control matrix narrows that interpretation. It selects the 13 of those 26 modules whose historical compiler wall time was below five seconds, then executes three adjacent build/import-only pairs per module, plus three `import Mathlib` controls. All 81 native processes return zero; original builds retain exact source bytes, and the controls retain only the original direct import lines. Both sides emit `.olean`, `.ilean` and `.c`. The sequential warm-OS-cache point uses a shared timing lock on CPUs 0–7, eight requested Lean workers and the 9,216 MiB native memory option. Its numbers are conditional on this short-module selection and instrumentation.

Per-module import-only wall medians range from 0.712 to 3.195 seconds, with a median of 1.355 seconds. The median per-module import/build ratio is 0.682; the ratio of summed module medians is 0.678. These are different descriptive aggregations, not a measured removable fraction or population estimate. The Mathlib-only control median is 3.002 seconds.

Conditional serial scale, not a library-cost estimate. For illustration only, suppose $n = 163{,}000$ separately launched calls each paid the same measured import-only median t:

$$H(t) = \frac{n\,t}{3{,}600} \quad \text{serial hours.}$$

The selected module medians give 32–145 hours, with 61 hours at the cohort median (unrounded: 32.23, 144.68 and 61.34 hours). The rounded n is a scale parameter, not a newly measured module count; the released source census remains 163,097. These are hypothetical repeated identical import costs under warm page cache, selected short successful modules and the measured observer. The range is the range of selected module medians, not an uncertainty interval for the population. Different import closures, cache states, dependency reuse and parallel execution prevent treating it as a full-build estimate or removable end-to-end time. No corresponding cold-cache or retained-process total has been measured.

Resident memory is mostly file-backed at this point (Table 4). Low reported `Shared_Clean` does not demonstrate that these pages cannot be shared: only one

Import-control observation	Value
Native runs / selected modules	39 / 13
Median native peak RSS	3.113 GiB
Median sampled PSS at RSS peak	3.100 GiB
Median file-backed fraction of PSS	92.89%
Median anonymous PSS at RSS peak	237.77 MiB
Sum of major page faults	0

Table 4: The 39 warm-cache import controls. Sampled PSS components use one simultaneous per-run RSS-peak sample; displayed medians need not add up. A 100 ms sample can miss peaks. Leader high-water RSS is a separate metric.

native compiler runs at a time, so mapped file pages can appear `Private_Clean`. A 3.113 GiB resident footprint is not evidence that 3.113 GiB of mutable heap was rebuilt. Actual cross-worker page sharing and environment-construction CPU still require separate measurements.

The observer is consequential. Native child CPU sums to 205.945 seconds and native wall intervals to 194.168 seconds. Driver waited-child accounting exceeds native CPU by approximately 54.045 CPU seconds, including observation and phase preparation. The runner scans process descendants and collects `smaps_rollup` at 100 ms intervals. This additional work is reported rather than subtracted from the native wall clocks. A completed follow-up changes descendant discovery while retaining 100 ms memory sampling. It executes three build/import repetitions on three historical short-module controls, plus three Mathlib-only repetitions: 21 successful native calls and 14.348 seconds of measured observer CPU alongside 49.964 native CPU seconds. Across the seven mode/module groups, follow-up/original wall-median ratios range from 0.920 to 1.087, including both slower and faster observations. This fixed-order, smaller-cohort comparison is neither an uninstrumented reference nor a randomized overhead estimate. Its observer CPU cannot be compared directly with the earlier 81-call matrix's 54.045-second aggregate.

Five separate import-only perf runs also complete. The user-plus-kernel Mathlib recording contains 579 samples and zero reported lost samples; its self-period shares include 9.29% for `lean_mark_persistent` and 5.69% for `lean_name_eq`. The other four recordings sample user cycles only and keep their own denominators. These profiles expose work within import preparation, not a cause of the complete library modules' system-CPU share. [**RESULT: Uninstrumented import controls, repeat-based observer uncertainty, cold-cache controls, allocation/CPU attribution and matched retained-state costs.**]

Replay costs depend on the checking boundary. The first compiled-module replay attempt stops before checking: all 28 targets and 13 empty controls fail module discovery. Those operational failures remain excluded from replay-cost statistics. After correcting artifact-path enumeration, a separate stock Lean 4.34.1 matrix completes all 15 requested import-relative target replays. Every successful invocation announces exactly its requested module and returns zero without diagnostics.

Module	Import-relative (s)	Fresh-scope (s)
Init Nat.Basic	2.026	2.428
Mathlib Nat.Basic	2.195	73.134
NearPinDeletion	2.556	censored at 180

Table 5: Separate bounded checker-process invocations with different trust/replay scopes. Fresh scope rechecks the loaded constant map, not only the target source; the last cell is not a successful checking time.

For the 13 short modules, median checker-process wall time is 3.510 seconds. The median ratio to each module's earlier three-repeat full-build median is 1.281. Thus eliminating source elaboration does not automatically make this fresh checker process cheaper in the selected cohort. Both durations include process setup and environment work, and the denominators include the earlier import experiment's observer. These are separate sequential captures, not contemporaneous randomized construction/replay pairs or an exclusive-kernel fraction. Grid and Amplitudes replay in 253.439 and 113.618 seconds, respectively; their ratios to separately retained historical standard builds are 0.400 and 0.630. These two historical comparisons remain separate from the 13-module median and do not establish service speedups.

Import-relative replay trusts imported declarations and rechecks target constants. In contrast, `--fresh` imports the target and replays the loaded constant map into an empty kernel environment. Both modes use the stock kernel and skip upstream-designated unsafe and partial declarations [17]; neither is a separately implemented checker or an axiom-clean proof audit. The three scope controls in Table 5 illustrate why their costs cannot be exchanged. Mathlib's Nat.Basic takes 2.195 seconds import-relative but 73.134 seconds with `--fresh`, a 33.32 ratio. NearPinDeletion's fresh-scope call reaches its 180-second wall bound; its 180.111-second observed duration is a censored attempt, not a completed replay cost, failed proposition or successful ratio. Fresh scope does not mean a cold OS page cache.

[**RESULT: Contemporaneous repeated build/replay comparisons, successful empty-module controls after repair, exclusive target-check cost and replay coverage beyond the selected 15 targets.**]

Established environment reuse provides a useful contrast [9, 25]. In one completed miniF2F resource comparison, four warm workers take 25,176.49 seconds and the

prepared engine takes 20,323.01 seconds. All 7,808 final status aliases and 3,014 accepted IDs match. The 1.23882 ratio includes retry ordering, affinity, native work, fallback and retirement; it does not isolate copy-on-write or import elimination. Raw payloads differ, and a labelled two-field normalization still leaves 23 rejected-source diagnostic differences. One prepared parent serves the ordinary frontier, while 42 fresh exclusive resource retries and 18 different-import fallbacks remain charged. This service comparison does not estimate a library build's import share.

System-design implication. Small modules can be dominated by process/environment preparation even when their proof bodies are cheap. This fixed cost motivates resident or prewarmed checking processes that retain a compatible imported environment across requests. State and options must remain request-correct, and incompatible imports still require separate environments or reloads. Import amortization is established engineering, not our novelty claim. Evaluate it against a competent warm service and charge replicated resident memory, incompatible contexts, reloads, cancellation and cleanup [19]. A gain over fresh processes alone is insufficient evidence of service benefit. The instrumented import matrix and limited observer follow-up establish substantial preparation-related latency in their selected cohorts, but not how much a production service can remove. The large file-backed share also prevents inferring memory-capacity gains by multiplying per-process RSS. Calibration and matched retained-state comparisons must establish these benefits without changing verification behavior. A verification service must also expose its checking boundary: rebuilding a source, rechecking it relative to trusted imports and replaying its full loaded constant map are different operations. Charge discovery, imports and cleanup for the chosen operation; do not substitute a cheaper trust scope for an equivalent performance comparison.

6 Budget-failure exposure is workload dependent

A request that ends at a resource limit consumes verification capacity but does not establish a false proposition or an inherently impossible proof. We therefore separate budget exposure, ordinary nonzero exits and slow-tail concentration.

In the retained miniF2F engine point, 85 final timeouts and 41 final resource failures identify 126 distinct sources. Their physical attempt sums, including retries, total 38,795.133 of 52,992.124 seconds: 73.21% of worker-attempt time. This is not 73.21% of the concurrent 20,323.01-second makespan. A separate fixed 42-source exclusive rerun under the aggregate budget returns 41 watchdog resource failures and one timeout, with no recovered acceptance. It does not prove that those computations fail under every environment or budget.

Point	Sources	Budget sources	Cost share
miniF2F attempts	7,808	126	73.21%
ProofNet pass 1	5,952	70	11.57%
EPFL completed prefix	136	12	25.60%
EPFL training selection	12	6	55.04%
APRIL mutations	39	16	50.53%
APRIL references	39	4	18.94%

Table 6: Budget-exposed source time under each point's own measurement boundary. miniF2F includes physical retries; other rows use request clocks. Selection, environments and limits differ. No row is a sound early-rejection rule or a cross-workload speedup comparison.

In the first retained ProofNet pass, 70 of 5,952 sources with budget diagnostics consume 138.011 of 1,192.435 request seconds, or 11.57%. The other two passes have their own clocks and are not pooled. The selected EPFL and APRIL points also vary (Table 6); they are small diagnostic samples, not population estimates.

AutoVerus illustrates a different distinction. In three complete direct passes, the slowest 58 of 3,566 sources (the ceiling of 1.6%) consume 3.399%, 3.338% and 3.315% of summed external process time. These are *slow-tail shares*, not budget-failure shares. No capture timeout is reported in those passes. Meanwhile, 3,376 nonzero process exits consume about 95% of each pass's time. Ordinary rejection can be common without a concentrated long tail. Its first-pass external, reported frontend and SMT sums are respectively 639.238, 443.690 and 63.120 seconds. SMT is initialization plus query interaction elapsed time, not isolated Z3 CPU. Missing timing JSON or SMT fields remain unavailable; nested timers are not added twice.

For a common slow-tail definition, the slowest 96 ProofNet requests consume 33.55% in its first pass. The miniF2F retained budget-source witnesses prove a lower bound of 72.68% for its true slowest 125 source attempt sums. This miniF2F value is a subset-based lower bound, not a complete top-125 ranking. Neither number is interchangeable with the budget-share column.

System-design implication. Failure-aware scheduling must be justified for the target workload and consumer. Exact all-candidate rewards still require every result. First-success consumers may cancel peers after a checked success, but an unsolved task still requires all candidates. Historical resource failures are not safe semantic cache entries without complete state, resource and replay obligations. A workload with a 73% budget-exposed attempt share invites different experiments from one whose slowest 1.6% consumes only 3.4%. Expose all-candidate and first-success modes explicitly in the service: their cancellation permissions and useful-work metrics differ. Provision and evaluate retry capacity

from that workload's measured resource tail, rather than transferring a timeout policy from another candidate stream. Generation cost also matters: AutoVerus publisher log labels spend 95.93% of aggregate logged span on inference, including waits/retries. The residual is unattributed, not a measured verification component, and cannot be combined with our later replay clocks.

7 Resource limits and profilers can change the observed answer

A verification verdict includes the executable environment, options, accepted context, resource policy and service interpretation. Lean recursion-depth, allocation-heartbeat and wall limits are distinct. A stored budget label is not necessarily reproduced by source text alone.

Six released EPFL exact-program controls pass under the reconstructed native context with original source, annotated source and an enlarged heartbeat setting. A recursion depth of 128 fails, whereas 256 and the default 512 pass. The original producer context is unavailable. This exposes a concrete sensitivity, not a proven publisher label bug. Replayed reference proofs, generated mutations and service responses must therefore preserve their separate provenance.

Observation can also change the work completed. A Lean 4.9.0-rc1 diagnostic executes 200 sources in control/observe pairs with the same diagnostic `maxHeartbeats=3000`. Both retain InfoTrees; observe additionally enables zero-threshold `trace.profiler`. Seven pairs change from a no-error flag to a budget error. Raw error arrays differ for 80 pairs. The control/observe body sums are 779.388 and 80.075 seconds; the shorter profiled time reflects earlier interruption, not acceleration. The complete process takes 1,170.883 seconds. A no-error frontend flag is not whole-source, axiom-clean service acceptance.

A later linked C-timing control preserves error flags, diagnostic arrays and InfoTrees on all 200 sources. Yet measured allocation-heartbeat deltas differ for 63 timing-off and 62 timing-on sources relative to stock. Its six serial processes take 3,081.953 seconds, exceeding the declared 1,800-second group limit; these are retained exploratory diagnostics. Even outcome equality does not establish exact budget accounting. Their 93.17% exclusive `eqRefl` body-CPU share cannot be transferred to the original service: one diagnostic call runs for 720.250 seconds inside that frame, while the service stops the request at 300 seconds.

System-design implication. Budgeted verification needs a versioned work-accounting contract. Define reproducibility-oriented cutoffs in documented logical work units where the verifier can meter them reproducibly; retain wall time and RSS as separate operational safety limits. Pin recursion depth, toolchain, worker settings and context rather than reporting only a timeout in seconds. Lean's allocation-based heartbeats are not automatically an instrumentation-invariant meter: the measured controls change those deltas even when diagnostic outcomes match. An improved budget policy must therefore validate counter and outcome stability, not merely replace a wall timeout with an existing heartbeat option. A profile also needs its own denominator and an explicit perturbation study. Hardware sampling, source tracing and extra kernel rechecks have different effects. Record changed outcomes before attributing stage cost, and compare raw diagnostics as well as acceptance sets. [**RESULT: Same-input privileged profiling repetitions, perturbation uncertainty and reproducibility across the generated-software workloads.**] Do not infer an accelerator ceiling or semantic equivalence by borrowing an unprofiled run's denominator.

8 Certificate repair helps selected pathological reductions

This case study tests an implication of the profiles: representation can matter more than performing the same arithmetic faster. It is not the paper's main contribution or a generic verified extraction compiler. All proposed replacements still require native checking.

A small finite-count proof groups 10,000 tests into 100 blocks and proves correspondence to the original predicate/cardinality. Its computation theorem has no axioms; the bridge has only standard logical axioms, without `sorryAx` or a native-evaluation axiom. A complete successful control takes 30.581 seconds, including a warm extra declaration check of 8.557 seconds. That subclock is not added again or compared with a separately timed untrusted numeric loop as a checking speedup.

The broader miniF2F test makes the coverage boundary explicit. For 64 exact sources in two selected pathological computation families, original execution has zero acceptances, 13 ordinary errors and 51 budget outcomes; the repair has 31 acceptances, 29 errors and four budget outcomes. A third, 32-source power family adds only two acceptances; 29 remain errors and one remains budget-limited. These are changed-source proof repairs with checked enclosing declarations and axiom audits. They are not a transparent unchanged-input verifier optimization. Combining these repairs with historical accepted IDs is not a fresh full-population run or a pass-set preservation experiment.

The contrast with modern library proofs is negative. Seven matched Int/list certificate comparisons accept in both conditions, but all seven certificate variants are slower. Whole-source sums increase from 13.318 to 23.937 seconds; additional warm declaration checks increase from 2.328 to 5.758 seconds. The extra checks exclude original construction and first checking, so they

do not isolate total kernel CPU. Further external literal/certificate tests recover no whole ProofNet, EPFL or APRIL source in the tested subsets. An intermediate tactic hit can still leave a later false or unproved obligation; closing that tactic is not acceptance of its enclosing source.

System-design implication. Selected deep reductions can benefit from a different certified algorithm, but that result does not generalize to every decision tactic or library. Certificate generation, correspondence proofs, first kernel checking, imports and complete-source acceptance must all be charged. The checked semantic boundary matters more than the size of a raw arithmetic loop. Select certified representation changes from the observed reduction family, then test transfer to untouched sources and charge every bridge obligation; a fast standalone evaluator is insufficient. Existing external search/check architectures and native decision procedures remain prior techniques [10, 11].

The same boundary limits device conclusions. In the completed miniF2F counterexample-screening comparison, exact post-checking refutes intermediate goals in 29 ordinary-error candidates, removes no timeout and increases whole elapsed time by 8.86%. It is a small component with limited measured coverage, not a general GPU acceleration result. Failure of that implementation does not prove GPUs cannot help a different profiled workload. The external arithmetic oracle remains a frozen baseline, not a contribution or a new experimental direction.

9 What the characterization supports, and what remains unknown

The observations support six design priorities: declare build coverage; attribute system work before adding cores; isolate the import floor; separate failure frequency from tail concentration; measure observer perturbation; and evaluate proof-representation changes through complete native checking. They do not yet establish one generally superior verifier.

There are three major open evidence boundaries. First, all-source native cost estimation is incomplete. Completed samples, purposeful stresses and dependency work cannot be pooled into a whole-library point estimate, and cold preparation costs remain outside the compiler clocks. Second, generated-software availability and performance must be established independently of published task scaffolds and benchmark scores. Third, current operating-system counters are observational; causal attribution awaits the privileged profiling campaign. No missing result is filled with an estimated speedup.

The artifacts preserve unsuccessful prototypes, signal-limited captures, budget interruptions and uninvoked dependencies. An earlier 8,031-unit state-reuse plan is interrupted rather than complete: 2,640 original records are archived, one panic record has unknown elapsed time and at least 1,273 later observed ledger entries are currently unrecoverable after storage eviction. Those missing records are neither ordinary failures nor evidence of zero cache opportunity. The present paper uses only retained data with declared scope.

Public corpus selection also limits generalization. EPFL's construction filters some infrastructure failures, invalid outputs and missing declarations; APRIL mutation subsets are not natural arrival traffic. AutoVerus source-sorted replay is not a historical arrival trace. Repository buildability, per-file compiler exit, tactic success and axiom-clean acceptance remain different outcomes. **[RESULT: Representative software audit, complete profile tables and repeat uncertainty required for the final characterization.]**

10 Related work

Proof-assistant runtime and build performance. Performance engineering of proof assistants predates AI-generated proofs. Lean's architecture and reference-counting work study efficient execution of functional metaprograms [4, 33]. Tebbi and Gross's Ltac profiler attributes nested tactic invocations in Coq [32]. Huch and Wenzel study distributed Isabelle/AFP builds and schedules for multithreaded sessions [7]. These establish runtime, tactic and dependency-scheduling costs as important problems. Our work measures which of these boundaries dominate particular generated artifacts; low realized core use does not by itself establish a new scheduling or runtime technique.

Mathlib compilation and continuous integration. Community profiling has already examined per-file build times and typeclass, tactic, import and checking phases in Mathlib [31]. We do not claim the first library cost profile. Mathlib's official cache downloads compiled artifacts keyed by toolchain, source and imported dependencies; CI constructs and publishes those artifacts [21]. This makes incremental CI, cached downstream builds and fresh source compilation different cost boundaries. Prior Isabelle-build work also discusses Mathlib's cached workflow in its contemporary setting [7]. We do not import its CI duration or community profiler percentages into our hardware/version denominators. Our new-source study distinguishes default-root coverage, successful artifact growth, CPU/system time and uncompleted probability draws rather than treating a cache hit or a build command's completion as all-source coverage.

Native replay and external proof checkers. The archived `lean4checker` is now distributed with Lean as `leanchecker`. It replays declarations through the official kernel: it is a separate checking pass, not a separately implemented kernel [17]. Carneiro's Lean4Lean reimplements the checker in Lean and evaluates post-compilation replay on major libraries [3]. Lean Kernel Arena evaluates multiple implementations on exported environments, including acceptance, rejection and corner cases [16]. Its instruction-derived virtual CPU scores and a local compiler wall clock have different meanings. These studies address checking performance and trust at post-export or compiled-environment boundaries. Our measurements additionally charge source elaboration, imports, emitted artifacts and failed candidate work; a faster external kernel is useful only for the share at its boundary. Axioms, admissions and native evaluation remain separate trust questions.

RL verification services and prepared proof state. Kimina Lean Server uses concurrent persistent REPLs and cached import headers for RL and proof feedback [25, 27]. Its published throughput study establishes import reuse as a practical service technique, not a new mechanism here. Snapshots retain live proof state for tactic search [29]; stock incremental elaboration reuses prior source state [13]. APE-Bench evaluates file/repository-level proof engineering with pinned library versions and shared infrastructure [36]. Such services and benchmarks establish why reusable contexts matter. Our comparison asks where resource-bounded time and memory go across research mathematics, software artifacts and candidate streams, and whether profiling or changed limits alter their measured verdicts. We do not transfer service speedups to a serial library build.

Reuse, scheduling and certified computation. Fine-grained verification caching [18], tactic/prefix reuse [1, 30], simplifier memoization [12], dynamic dependencies [26, 6] and age-based scheduling [20, 28] are established alternatives. The certificate case study tests a specific consequence of a measured reduction bottleneck; external search with native checking is prior art.

The gap addressed here is a boundary-consistent characterization of released AI-generated formal artifacts, with a research-level library as the primary population and rejected/resource-limited attempts retained as contrasts. It joins source/build coverage to native costs, separates mapped resident pages from mutable state, and tests whether observation preserves outcomes. It complements build engineering and checker benchmarks rather than claiming that their bottlenecks are new. Complete software and causal-profile findings remain explicit open results.

11 Conclusion

Verification of AI-generated formal artifacts has workload- and boundary-dependent costs. The research-library evidence exposes substantial system CPU, low realized core use, a repeated fresh-process floor and incomplete default-build coverage. Candidate streams expose different failure and tail structures; resource limits and profiling can change the observed outcome. Targeted certificate repair illustrates an algorithmic response to selected pathological reductions, while its negative transfer results limit that response's scope. **[RESULT: Final generated-software, calibrated import-cost, perf-cause and population-cost findings.]** The characterization argues for measuring the complete acceptance path and its actual workload before selecting an acceleration mechanism.

References

[1] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. The tactician (extended version): A seamless, interactive tactic learner and prover for coq, 2020. Extended version of the CICM paper.

[2] Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, et al. A benchmark for vericoding: formally verified program synthesis, 2025. Output tree inspected at 387cd69996792d452ead7b0460f36ee4c5cdd148.

[3] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2025. Version 3, revised 14 September 2025; kernel replay, not source-compilation timing.

[4] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[5] EPFL-LARA. Lean evaluation benchmark, 2026. Pinned release 0bbf31331bd1c726f539b311cce8b058a821f15c; accessed 30 October 2026.

[6] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable, demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 156–166, 2014.

[7] Fabian Huch and Makarius Wenzel. Distributed parallel build for the isabelle archive of formal proofs. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPIcs*, pages 22:1–22:19, 2024.

[8] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 2023.

[9] Lean community. Lean community repl, 2026. Accessed 2026-10-07; year denotes access year.

[10] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[11] Lean contributors. Lean 4.9.0-rc1 ordinary decision tactic implementation. Pinned evalDecide handler; accessed 30 October 2026.

[12] Lean contributors. Lean4.22.0 release: simplifier cache reuse, 2025. Simp section; PR 8880.

[13] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[14] Lean contributors. Lean 4.34.1 auxiliary-lemma implementation, 2026. Branch-local cache lookup; accessed 30 October 2026.

[15] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026. doKernel invokes mkAuxLemma; accessed 30 October 2026.

[16] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[17] Lean contributors. lean4checker and the built-in leanchecker, 2026. Official archived README; integrated into Lean from v4.28.0; accessed 31 October 2026.

[18] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[19] Linux man-pages project. fork(2) linux manual page, 2026. Accessed 2026-10-07; year denotes access year.

[20] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[21] Mathlib contributors. Mathlib build cache and ci uploads, 2026. Official cache README and Cache/CI.md, accessed 31 October 2026; general CI context, not measured CI cost.

[22] Meta Platforms. Lean Kernel Verify: Compiler-based safety checker for gpu kernels, 2026. Pinned release 129e65bfd8efbf6c45710732d205c941ef65928c; accessed 30 October 2026.

[23] Microsoft Research and AutoVerus contributors. AutoVerus: Artifact evaluation, 2026. Source revision cbf9c0c6337b224fd8e5b7cb4e01ae65c0f98bc1; accessed 16 October 2026; year denotes access year.

[24] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[25] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[26] Salsa contributors. The red-green algorithm, 2026. Official algorithm documentation; accessed 30 October 2026.

[27] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[28] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. SOAP: One clean analysis of all age-based scheduling policies. arXiv:1712.00790v2, 2018.

[29] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[30] SHRED contributors. SHRED: Reuse repeated work across thousands of Lean proof attempts, 2026. Software and experimental artifact; revision a6616e99534ce777dcebfd56566eaaffb63997a8; accessed 16 October 2026.

[31] Michael Stoll. Ways to speed up mathlib. Lean community Zulip archive, 25 April 2024, 2024. Community measurement report; no timing or phase percentage transferred to our experiments.

[32] Tobias Tebbi and Jason Gross. A profiler for ltac. CoqPL Workshop abstract, 2015.

[33] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Proceedings of the International Symposium on Implementation and Application of Functional Languages (IFL '19)*, 2020.

[34] UW Math AI. APRIL: Lean proof repair dataset, 2026. Pinned release 4e3075279b4aedfb28fe435d08a8caf76272586d; accessed 30 October 2026.

[35] Vericoding contributors. Vericoding: released model-tagged programs and final experiment tables. Artifact repository, revision b265a3b2bce31e09d264306f419e810d02f98f44, 2026. Inspected 31 October 2026; final outputs do not constitute complete verification trajectories.

[36] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. Ape-bench: Evaluating automated proof engineering for formal math libraries, 2026. Version 3, revised 2 February 2026.

[37] Zhe Ye, Hantao Lou, Yuechun Sun, et al. Vero: Can ai agents build formally verified software repositories?, 2026. Official repository inspected at 0a7325df9e9e6dbc275c0ad483b3d1cbe38d9b09; evaluated run outputs not established.